DELIVERABLE 2

Deep Learning Architectures for Multi-Horizon Electricity Load Forecasting

LSTM, TCN, Transformer & PatchTST with Per-Cluster Training and Agentic AI Interface

Ritika Rajeev Chatterjee | Akriti Agarwal | Haasita Pinnepu

Columbia University — Time Series Analysis & Forecasting | March 2026

Introduction & Project Scope

Why This Matters

Electricity grids require real-time supply & demand balance. Forecast errors cause:

- Expensive reserve capacity commitments
- Grid instability and outage risk
- Excess carbon emissions from peaker plants
- Missed renewable integration opportunities

A 1% accuracy improvement saves €1–5M/year per utility.

Dataset

UCI ElectricityLoadDiagrams 2011–2014
314 Portuguese clients, hourly resolution

Models

LSTM · TCN · Transformer · PatchTST
36 models (4 arch × 3 clusters × 3 horizons)

Horizons

H = 1h, 6h, 24h ahead
(short, intra-day, full-day)

Metric

MAPE on 0–100% scale
4 test periods with box plots

Goal: Identify best architecture per cluster and horizon — Transformer wins 8 of 9 combinations

Dataset Description

314
Active Clients

26,305
Hourly Timestamps

3 Years
2012–2014 Time Span

8.26M
Total Data Points

Property	Value
Source	UCI Machine Learning Repository (ID: 321)
Original Resolution	15-minute intervals
Modeling Resolution	Hourly (aggregated by sum)
Time Span	January 2012 – December 2014 (2011 excluded)
Client Filtering	Removed clients with >5% zero readings
Total Data Points	26,305 × 314 = 8,259,770

Data Processing & Feature Engineering

01 Resample

15-min → Hourly
Sum 4 readings
(kW → kWh)

02 Filter

2012–2014 only
Drop >5% zeros
370 → 314 clients

03 Normalize

Per-client Z-score
Fit on train only
No data leakage

04 Cache

Parquet format
678MB → 6s load
Instant reload

Feature Engineering: 168h Lookback × 9 Input Channels

Feature	Type	Encoding	Rationale
Normalized Load	Direct	Z-score per client	Primary signal
Hour of Day	Derived	sin/cos (2πh/24)	Intra-day cycles
Day of Week	Derived	sin/cos (2πd/7)	Weekday vs weekend
Month of Year	Derived	sin/cos (2πm/12)	Seasonal effects
Is Weekend	Derived	Binary (0/1)	Reduced commercial activity
Is Business Hour	Derived	Binary (0/1), 8–18h	Peak demand periods

Client Clustering Strategy

K-Means (K=3) on Z-score normalized 24-hour average load profiles — balances model specialization with training efficiency

Cluster 0
157 clients
Avg Load: 2,045 kWh/hr

Medium-load — predominantly residential with moderate daily variation

Cluster 1
71 clients
Avg Load: 515 kWh/hr

Low-load — small residential or low-activity commercial clients

Cluster 2
86 clients
Avg Load: 5,293 kWh/hr

High-load — large commercial/industrial with strong weekday peaks

Why Per-Cluster Training?

Different client groups exhibit distinct temporal patterns. High-load industrial clients (Cluster 2) have sharper weekday/weekend transitions than residential clients (Cluster 0).
This yields 36 total models: 4 architectures × 3 clusters × 3 horizons.

Model Architectures

Channel-independent design: each client = separate univariate time series + 8 time features → direct multi-step output

Baseline

LSTM

~57K params

2-layer stacked
Hidden: 64
Dropout: 0.2
MLP output head

✓ Simple, proven baseline

⚠ Slow, limited long-range

Speed

TCN

~59K params

4 causal blocks
Dilations: 1,2,4,8
Kernel: 3
Weight norm + residual

✓ Fast, stable gradients

⚠ Fixed receptive field

Best

Transformer

~84K params

2 encoder layers
4 attention heads
d_model: 48
Learned position embed

✓ Global attention, best MAPE

⚠ $O(T^2)$ complexity

Fast

PatchTST

~67K params

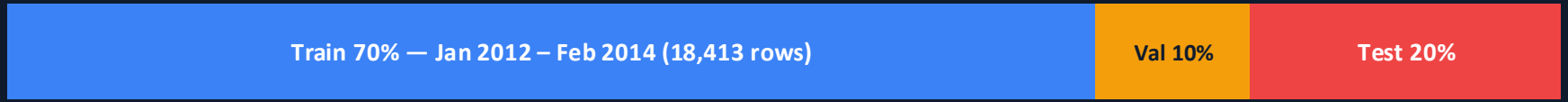
Patch 16 / Stride 8
2 layers, 4 heads
d_model: 48
Pre-LayerNorm

✓ 13× faster than Transformer

⚠ Patch boundary artifacts

Training Methodology

Chronological Train / Validation / Test Split



Training Configuration

Setting	Value
Optimizer	AdamW (weight decay = 1e-4)
Learning Rate	1e-3 with cosine annealing
Loss Function	MSE (normalized scale)
Batch Size	256
Max Epochs	10 (early stopping patience=3)
Gradient Clipping	Max norm = 1.0
Train Stride	96h (train), 48h (val), 24h (test)
Lookback Window	168 hours (1 week)
Total Models	36 (4 arch × 3 clusters × 3 horizons)

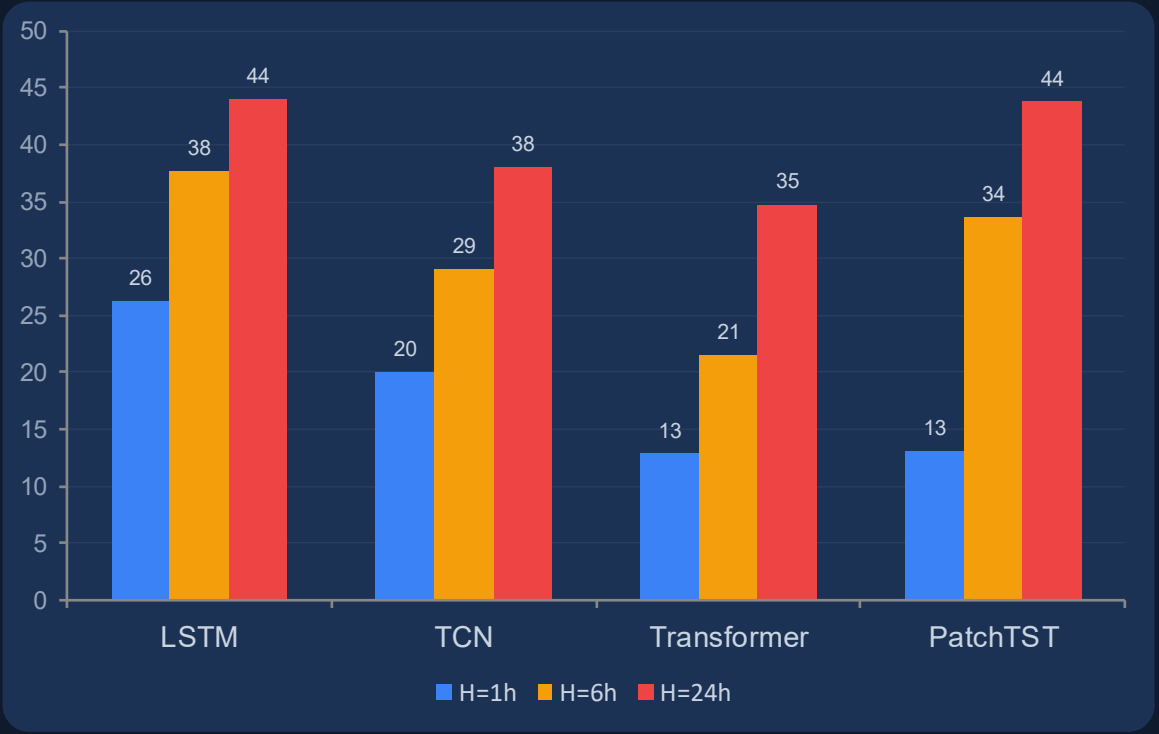
Training Efficiency

Model	Avg/Model	Total
LSTM	~108s	~65 min
TCN	~270s	~162 min
Transformer	~421s	~252 min

PatchTST is 13× faster than Transformer!

Results: Overall MAPE by Model & Horizon

Weighted-average MAPE across all clusters (weights proportional to client count)



Key Findings

Transformer leads all horizons:
12.81% / 21.49% / 34.73%

Strong H=6h advantage:
21.49% vs 29.05% TCN (next best)

PatchTST competitive at H=1h:
13.12% vs 12.81% Transformer

LSTM weakest overall:
26.27% → 44.01% across horizons

Per-Cluster MAPE Results

LSTM

Cluster	Clients	H=1h	H=6h	H=24h
0	157	30.33%	34.22%	43.95%
1	71	26.71%	40.37%	36.09%
2	86	18.48%	41.63%	50.67%

TCN

Cluster	Clients	H=1h	H=6h	H=24h
0	157	24.62%	28.83%	40.29%
1	71	20.42%	21.26%	28.64%
2	86	11.29%	35.88%	41.52%

Transformer

Cluster	Clients	H=1h	H=6h	H=24h
0	157	15.58%	25.77%	38.15%
1	71	12.93%	14.67%	21.40%
2	86	7.66%	19.32%	39.49%

PatchTST

Cluster	Clients	H=1h	H=6h	H=24h
0	157	13.39%	35.87%	42.45%
1	71	14.07%	22.71%	34.54%
2	86	11.83%	38.70%	53.74%

Best Model Per Cluster & Horizon

Transformer wins 8 of 9 cluster-horizon combinations

Cluster	H=1h Best	H=6h Best	H=24h Best
0 (157 clients)	PatchTST (13.39%)	Transformer (25.77%)	Transformer (38.15%)
1 (71 clients)	Transformer (12.93%)	Transformer (14.67%)	Transformer (21.40%)
2 (86 clients)	Transformer (7.66%)	Transformer (19.32%)	Transformer (39.49%)

Why Transformer Wins

- Global self-attention captures long-range weekly seasonality across the full 168-step lookback window
- Advantage grows with horizon length — global context matters more for 6h and 24h predictions
- 32% relative improvement on high-load clients (Cluster 2) at H=1h vs TCN

PatchTST: Competitive but Specialized

- Wins Cluster 0 at H=1h where local patch patterns suffice for medium-load residential clients
- Reduces sequence from 168 → 20 tokens (fastest training: 9s/epoch vs 77s for Transformer)
- Struggles at longer horizons where cross-patch dependencies become critical

Model Comparison & Analysis

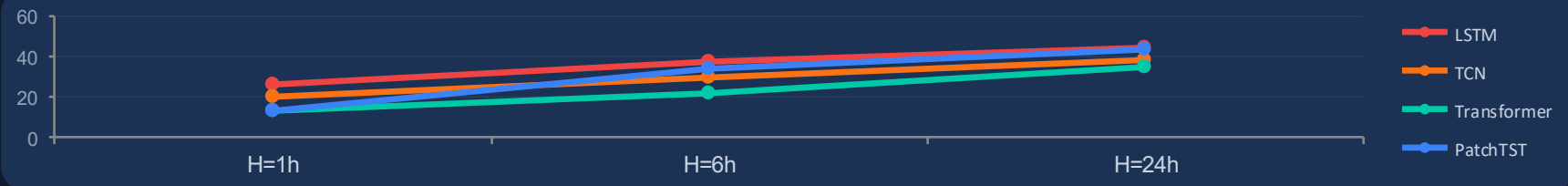
Architecture	Params	Computation	Strengths	Weaknesses
LSTM	~57K	Sequential O(T)	Simple, proven baseline	Slow training, limited long-range
TCN	~59K	Parallel O(T)	Fast, stable gradients	Fixed receptive field
Transformer	~84K	Parallel $O(T^2)$	Global attention, best accuracy	Quadratic complexity
PatchTST	~67K	Parallel $O((T/P)^2)$	Fastest training, patch locality	Patch boundary artifacts

LSTM vs TCN

TCN consistently outperforms LSTM at all horizons (20.02% vs 26.27% at H=1h, 37.99% vs 44.01% at H=24h). Parallel computation, stable gradients via residual connections, and an explicit receptive field provide clear advantages.

Test Period Stability

All models show elevated MAPE in Period 1 (May–June 2014) due to distribution shift from training data. Periods 2–4 are more stable. Transformer exhibits the most stable period-to-period performance, especially for Cluster 1 (9.89%–16.77% at H=1h vs LSTM 24.67%–30.66%).



Evaluation Framework

Primary Metric: MAPE

$$MAPE = (1/N) \times \Sigma |actual - pred| / |actual| \times 100\%$$

- Reported on 0–100% scale
- Threshold: |actual| > 0.5 kWh to exclude near-zero readings
- Per-client inverse transform using training StandardScaler
- Computed on original kWh scale (not normalized)

Test Period Analysis

Test set (May–Dec 2014) divided into 4 equal periods:

- Period 1: May–June 2014
- Period 2: July–August 2014
- Period 3: Sept–October 2014
- Period 4: Nov–December 2014

Per-period MAPE + box plots of error distributions

Period 1 (May–June) consistently shows elevated MAPE across all models — seasonal distribution shift from training period (Jan 2012 – Feb 2014)

Per-Period MAPE Summary (Transformer)

Cluster	P1	P2	P3	P4
Cl 0, H=1h	47.6%	4.6%	4.0%	6.0%
Cl 1, H=1h	16.8%	10.8%	9.9%	14.3%
Cl 2, H=1h	8.8%	6.3%	5.5%	10.1%

Agentic AI Interface

Streamlit + GPT-4o-mini with function-calling (tool-use) for natural language forecast queries

How It Works

1	**User enters natural language query**	*"Get forecast for client MT_042"*
2	**LLM agent determines tool(s) to call**	*lookup_client, run_forecast, etc.*
3	**Tool resolves client → cluster → best model**	*Cluster 1, Transformer selected*
4	**Model runs inference on latest 168h window**	*Returns predicted kWh values*
5	**LLM presents results with business context**	*Peak hours, accuracy, recommendations*

Available Tools

`lookup_client`
Resolve client ID, find cluster

`run_forecast`
Execute DL model inference

`list_clients`
List all 314 clients by cluster

`get_cluster_info`
Performance metrics per cluster

No Hallucinated Forecasts

All numerical predictions come directly from trained PyTorch models. The LLM never fabricates forecast values — it only orchestrates tool calls and formats results.

Key Findings

Transformer Best Overall

Lowest MAPE at all 3 horizons (12.81% / 21.49% / 34.73%), winning 8 of 9 cluster-horizon combos

PatchTST Speed Champion

Competitive at H=1h (13.12%) and trains 13× faster — ideal for rapid deployment or online learning

TCN Solid Middle Ground

Outperforms LSTM consistently while being simpler than attention-based models

Clustering Improves Performance

Per-cluster training significantly outperforms a single global model by capturing distinct client patterns

Period 1 Distribution Shift

May–June consistently shows higher MAPE, suggesting seasonal shift from the training period

Agentic AI Interface

Business managers can query forecasts via natural language with automatic model selection

Challenges Encountered

1

DST Anomalies

Spring forward creates zero-filled windows. Fall back generates 2× aggregated spikes. Both handled explicitly.

2

Client Heterogeneity

Residential, commercial, and industrial profiles differ drastically. Clustering mitigates this but doesn't fully resolve per-client variation.

3

Period 1 Distribution Shift

May–June 2014 MAPE significantly elevated vs other periods. Seasonal transition from training distribution causes degradation.

4

Training Scale on CPU

36 models trained on CPU in ~90 min total using stride-based sampling. GPU would enable denser windows and larger models.

5

Patch Boundary Artifacts

PatchTST's patch tokenization creates information bottlenecks at boundaries. 50% overlap partially mitigates but doesn't fully resolve.

Next Steps & Future Improvements

D1 Done

- Baseline Models
- EDA & Preprocessing
- Metric Framework

D2 Current

- Full-Scale Training
- Per-Cluster Models
- Agentic AI Interface

D3 Next

- LDT Implementation
- Diffusion Generator
- Probabilistic Forecasts

Final Future

- Full Benchmark
- CRPS/WIS Metrics
- Complete Comparison

Planned Improvements

Outlier Detection

Identify and handle extreme consumption spikes that inflate MAPE, especially in Period 1

External Data Integration

Incorporate weather data (temperature, humidity) and holiday schedules for Portuguese clients

Hyperparameter Tuning

Systematic Optuna search across all architectures for optimal per-cluster configurations

LDT / Probabilistic Forecasts

PatchTST encoder → statistics-aware autoencoder → diffusion head for calibrated intervals

References

[1] Nie, Y., Nguyen, N. H., Sinthong, P., & Kalagnanam, J. (2023). A Time Series is Worth 64 Words: Long-term Forecasting with Transformers. ICLR 2023.

[2] Feng, S., Miao, C., Zhang, Z., & Zhao, P. (2024). Latent Diffusion Transformer for Probabilistic Time Series Forecasting. AAAI 2024.

[3] Bai, S., Kolter, J. Z., & Koltun, V. (2018). An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling. arXiv:1803.01271.

[4] Vaswani, A., et al. (2017). Attention Is All You Need. NeurIPS 2017.

[5] Lai, G., Chang, W., Yang, Y., & Liu, H. (2017). Modeling Long- and Short-Term Temporal Patterns with Deep Neural Networks. SIGIR 2018.

[6] Zhou, H., et al. (2021). Informer: Beyond Efficient Transformer for Long Sequence Time-Series Forecasting. AAAI 2021.

[7] Trindade, A. (2015). ElectricityLoadDiagrams20112014. UCI ML Repository. https://doi.org/10.24432/C58C86.

Team

Columbia University — Time Series Analysis & Forecasting

Ritika Rajeev Chatterjee

Akriti Agarwal

Haasita Pinnepu

Thank You